

ViewPager 及替代方案介绍

ViewPager 详细介绍

1. 什么是 ViewPager?

ViewPager 是 Android 官方提供的一个布局管理器，用于实现左右滑动切换页面的效果（如引导页、Tab 切换、轮播图等）。它属于 `androidx.viewpager.widget` 包（原 `android.support.v4.view`），是 `ViewPager2` 出现前的主流滑动容器。

2. 核心特性

- **支持页面滑动切换**：通过触摸事件或代码控制页面切换。
- **结合 PagerAdapter**：需要自定义 Adapter 管理页面数据（如 `PagerAdapter`、`FragmentPagerAdapter`、`FragmentStatePagerAdapter`）。
- **支持页面缓存**：默认缓存当前页左右各 1 页（可通过 `setOffscreenPageLimit()` 调整）。
- **提供页面切换动画**：可通过 `setPageTransformer()` 设置切换动画。

3. 使用步骤

Step 1: 布局文件中添加 ViewPager

Code block

```
1 <androidx.viewpager.widget.ViewPager
2     android:id="@+id/viewPager"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent" />
```

Step 2: 自定义 Adapter

- **FragmentPagerAdapter**：适合少量固定 Fragment 场景（如 Tab 页面），Fragment 实例会被缓存。
- **FragmentStatePagerAdapter**：适合大量 Fragment 场景（如长列表），会销毁不显示的 Fragment 以节省内存。

示例（FragmentPagerAdapter）：

Code block

```
1 public class MyPagerAdapter extends FragmentPagerAdapter {
2     private List<Fragment> fragmentList;
```

```
3
4     public MyPagerAdapter(FragmentManager fm, List<Fragment> fragments) {
5         super(fm, BEHAVIOR_RESUME_ONLY_CURRENT_FRAGMENT);
6         this.fragmentList = fragments;
7     }
8
9     @Override
10    public Fragment getItem(int position) {
11        return fragmentList.get(position);
12    }
13
14    @Override
15    public int getCount() {
16        return fragmentList.size();
17    }
18 }
```

Step 3: 绑定 ViewPager 和 Adapter

Code block

```
1 ViewPager viewPager = findViewById(R.id.viewPager);
2 List<Fragment> fragments = Arrays.asList(new Fragment1(), new Fragment2(), new
  Fragment3());
3 MyPagerAdapter adapter = new MyPagerAdapter(getSupportFragmentManager(),
  fragments);
4 viewPager.setAdapter(adapter);
```

4. 常用方法

- `setCurrentItem(int position)`：切换到指定页面。
- `addOnPageChangeListener()`：监听页面切换事件（滑动状态、页面选中、滚动位置）。
- `setOffscreenPageLimit(int limit)`：设置预加载页面数量。
- `setPageTransformer()`：设置页面切换动画（如深度动画、缩放动画）。

5. 局限性

- **不支持垂直滑动**：需自定义修改或使用第三方库。
- **Adapter 设计不够灵活**：与 Fragment 耦合度高，数据更新需手动处理。
- **性能问题**：大量页面时内存占用较高（FragmentStatePagerAdapter 可缓解但仍有局限）。

ViewPager2（官方替代方案）

ViewPager2 是 ViewPager 的升级版，基于 `RecyclerView` 实现，解决了 ViewPager 的诸多痛点：

- **支持垂直滑动**：通过 `setOrientation(ViewPager2.ORIENTATION_VERTICAL)` 设置。
- **基于 RecyclerView**：复用 ViewHolder，性能更优，支持 DiffUtil 高效更新数据。
- **兼容 Fragment**：使用 `FragmentStateAdapter` 替代原有 Adapter，生命周期管理更完善。
- **支持 RTL（从右到左）布局**。

使用示例：

Code block

```
1 <androidx.viewpager2.widget.ViewPager2
2     android:id="@+id/viewPager2"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent" />
```

Code block

```
1 ViewPager2 viewPager2 = findViewById(R.id.viewPager2);
2 viewPager2.setAdapter(new FragmentStateAdapter(this) {
3     @Override
4     public Fragment createFragment(int position) {
5         return fragments.get(position);
6     }
7
8     @Override
9     public int getItemCount() {
10         return fragments.size();
11     }
12 });
```

第三方替代方案

1. SmartTabLayout（与 ViewPager 配合）

- **功能**：提供可定制的 Tab 栏，支持与 ViewPager/ViewPager2 联动。
- **特点**：支持图标、文字、角标，可自定义指示器样式。
- **依赖**：

Code block

```
1 implementation 'com.ogaclejapan.smarttablayout:library:2.0.0@aar'
```

```
2 implementation 'com.ogaclejapan.smarttablayout:utils-v4:2.0.0@aar' // 适配
ViewPager
```

2. Banner（轮播图专用）

- **适用场景：**轮播广告、图片展示（替代 ViewPager 实现轮播）。
- **代表库：**
 - **Banner** (<https://github.com/youth5201314/banner>)：支持无限轮播、指示器定制、页面切换动画。

Code block

```
1 implementation 'io.github.youth5201314:banner:2.2.2'
```

- **ConvenientBanner：**轻量级轮播库，支持自定义布局。

3. ViewPagerIndicator（经典指示器）

- **功能：**为 ViewPager 提供圆点、线条、文字等指示器样式。
- **依赖：**

Code block

```
1 implementation 'com.viewpagerindicator:library:2.4.1@aar'
```

4. MagicViewPager（增强版 ViewPager）

- **功能：**支持垂直滑动、3D 画廊效果、卡片堆叠效果。
- **特点：**扩展了 ViewPager 的动画和交互能力。

5. CoordinatorLayout + TabLayout（官方组合）

- **功能：**与 ViewPager/ViewPager2 配合实现 Material Design 风格的折叠标题栏、Tab 联动。
- **使用场景：**APP 首页、详情页等复杂布局。

总结

- **简单场景：**优先使用官方 `ViewPager2`（替代 ViewPager）。
- **轮播需求：**选择 Banner 等专用库（更高效）。
- **复杂 Tab 交互：**结合 SmartTabLayout 或官方 TabLayout。
- **特殊动画/交互：**考虑 MagicViewPager 等增强库。

第三方库可根据具体需求选择，优先考虑维护活跃、文档完善的库以保证稳定性。

（注：文档部分内容可能由 AI 生成）